UNIDAD #6: DESPLIEGUE DEL SISTEMA.

El despliegue es el proceso estructurado de transferir, configurar y habilitar una versión estable del software en un entorno operativo (servidor o plataforma de nube), garantizando que el servicio sea accesible, escalable y mantenible para los usuarios finales.

6.1. El contexto

No todo el software funciona de la misma manera, cada programa tiene sus particularidades de usabilidad y llega a los usuarios de distintas formas, según sea el enfoque que tenga el programador basándose en el hardware donde se implementara dicha arquitectura, esto va a definir el tipo de despliegue que la aplicación va a necesitar para lograr llegar a los usuarios que pretende:

- **Aplicaciones Frontend/Clientes Estáticos (SPA):** Archivos estáticos inmutables que no requieren un entorno de ejecución activo en el servidor. Se distribuyen mediante Redes de Distribución de Contenido (CDN) para optimizar la latencia.
- **Servidores Backend:** Aplicaciones dinámicas que requieren un entorno de ejecución continuo (Runtime como Node.js, JVM o entornos nativos como Rust) acoplados a sistemas de gestión de bases de datos con estado continuo.
- **Apps de escritorio o móviles:** según sea el caso, dependerá de la forma en la que esta aplicación necesitara ser usada, ya que en ocasiones funciona localmente, en otras ocasiones, tienen dependencia de servidores externos, sin embargo, siguen el mismo principio al instalarse, mediante un ejecutable (.exe, APK, etc.) dentro del sistema operativo en cuestión.
- **Apps para desarrolladores:** estas son especiales, porque tienen que funcionar de forma precisa, sin afectar el trabajo de los desarrolladores, en ocasiones se hacen mediante comandos que instalan los componentes necesarios.

6.2. Análisis del estado inicial

Antes de tocar una sola línea de código de infraestructura, hay que entender dónde el sistema será desplegado, existen dos contextos en los que se pueden encontrar, cada uno con sus elementos y formas para gestionar el despliegue de forma precisa, con la finalidad de evaluar el riesgo de transición.

- **Greenfield:** Despliegue desde cero, el reto de la recolección, limpieza y carga de datos históricos (de Excel o papel a bases de datos relacionales/no relacionales) o simplemente que los usuarios se interesen en ella para su utilización.
- **Brownfield:** Reemplazo de sistemas activos, el reto de la convivencia, la migración en caliente y la tolerancia cero al downtime.

6.3. Estrategia de transición

Según sea el caso, se debe implementar una estrategia de transición, por lo cual es imperativo que determinen, el contexto, establezcan sobre que van a trabajar y poder determinar el cómo lo van a hacer, mediante algunas estrategias que pueden servir de guía en el proceso.

- **Tradicionales:**
  - **Big Bang:** Sustitución inmediata y total del sistema anterior por el nuevo. Alto riesgo técnico por la ausencia de un entorno de pruebas bajo carga real, pero de bajo costo operacional si el sistema no es crítico.
  - **Paralelo:** Ambos sistemas funcionan a la vez por algún tiempo, es seguro, porque mantiene la integridad de ambos sistemas mientras se hace la migración, tiende a ser más costoso al mantener dos arquitecturas funcionando a la par.
  - **Faseado (Gradual):** Se introduce por módulos o departamentos, poco a poco se va integrando dentro de la infraestructura de la organización si forzar a un cambio directo.
  - **Prueba piloto:** Se hace el despliegue para un grupo de usuarios selectos antes de hacer el lanzamiento global, este puede combinarse con la estrategia faseado.

- **Modernas:**
    - **Blue-Green Deployment (conmutación por balanceador):** es una técnica de despliegue continuo donde operan dos entornos de producción idénticos (Azul y Verde). Un balanceador de carga redirige instantáneamente todo el tráfico de los usuarios del entorno antiguo (Azul) al nuevo (Verde) tras probarlo, permitiendo cero tiempos de inactividad y un *rollback* (reversión) inmediato.
    - **Canary Deployment (Despliegue porcentual controlado):** Es una estrategia de lanzamiento de software que reduce el riesgo al implementar una nueva versión de una aplicación. En lugar de actualizar todo el entorno de producción a la vez, los cambios se exponen de forma gradual y controlada a un pequeño subconjunto de usuarios antes de lanzarlos al resto.

6.4. Configuración y Seguridad del Entorno

El “en mi maquina funciona”, debe morir aquí, el entorno de producción debe ser agnóstico y seguro.

- **Aislamiento estricto:** se debe establecer un ciclo de vida de los entornos (Desarrollo -> Preproducción -> Producción), de este modo si algo falla, que falle en un entorno controlado donde pueda ser gestionado de forma eficiente.
- **Gestión de secretos y configuración:** Inyección de configuración mediante Variables de Entorno, por lo cual se debe establecer una prohibición absoluta de credenciales y llaves de API en el control de versiones.

6.5. Empaquetado

El software ya no se despliega "desnudo" en el servidor, se despliegan artefactos inmutables.

- **Contenedores con Docker:** buscamos asegurar la paridad entre entornos.
- **Optimización Multi-stage:** Separación de la etapa de construcción (pesada, con compiladores y dependencias de

desarrollo) de la etapa de ejecución (ligera, segura y minimalista) (Build -> Artefact).

- **Filosofía del Artefacto según el Lenguaje:**
  - **Caso compilado (Rust):** Enlazado estático vs. dinámico (glibc vs. musl). Creación de imágenes microscópicas y seguras (uso de bases como scratch o alpine).
  - **Caso interpretado/transpilado (TypeScript):** Compilación obligatoria a JavaScript nativo para producción, purga radical de la carpeta node_modules (separación estricta de dependencies y devDependencies).
- **Seguridad del Contenedor:** Ejecución de procesos con usuarios sin privilegios del sistema (non-root users).

6.6. Automatización del Pipeline (CI/CD):

El despliegue manual es el enemigo número uno del éxito, todo paso a producción debe ser aburrido por lo predecible que es.

- **Integración Continua (CI):** Automatización de Linters, formateadores y pruebas unitarias/integración en cada *Push* o *Pull Request*.
- **Despliegue Continuo (CD):** Automatización de la construcción del contenedor, subida al registro de imágenes y actualización del servidor de producción de forma desatendida.

6.7. Observabilidad básica:

El despliegue no termina cuando el código sube; termina cuando se comprueba que funciona bajo carga real.

- **Monitoreo Esencial:** Lectura de métricas críticas de infraestructura (CPU, Memoria, Almacenamiento, Latencia de Red).
- **Gestión de Logs:** Centralización de logs de errores para auditoría interna. Saber que el sistema falló antes de que el cliente llame a quejarse.

- **Alertas y notificaciones:** En caso de que se presente una falla, debe ser notificada en la brevedad posible, para poder gestionar lo sucedido y mitigar los daños.

6.8. Gestión de catástrofes y resiliencia del sistema

No basta solamente con planificar el despliegue, es importante saber con exactitud que hacer en caso de que suceda alguna situación, por eso es recomendable establecer una serie de pasos que se puedan seguir para que la aplicación, si llega a fallar, dicha falla sea mitigada, algunas de estas pueden ser:

- **Las Métricas del Desastre (RTO y RPO)**
  - **RTO (Recovery Time Objective / Objetivo de Tiempo de Recuperación):** ¿Cuánto tiempo puede estar el sistema caído antes de causar pérdidas catastróficas? (Ej. 5 minutos vs. 2 horas).
  - **RPO (Recovery Point Objective / Objetivo de Punto de Recuperación):** ¿Cuánta data nos podemos permitir perder en caso de un fallo crítico? (Ej. Los últimos 10 minutos de transacciones o las últimas 24 horas).
- **Mecanismos de Auto-recuperación (Self-Healing)**

  Los sistemas modernos no esperan a que un humano despierte a las 3:00 AM para reiniciar un servidor colapsado, se implementan:
  - **Health Checks (Liveness y Readiness Probes):** Configuraciones en el servidor (o en el contenedor Docker) que realizan peticiones automáticas cada “X” segundos a una ruta específica del backend (ej. /health). Si la app no responde o devuelve un error, el orquestador sabe que el proceso está congelado.
  - **Políticas de Reinicio Automático:** Uso de administradores de procesos (como PM2 en TypeScript) o políticas de Docker (restart: unless-stopped) que levantan el proceso instantáneamente tras un fallo crítico (panic en Rust o uncaughtException en TS).
- **Estrategias de Rollback Automatizado y Manejo de Estado:**

- Si el monitoreo detecta que el nuevo despliegue eleva la tasa de errores HTTP al 500 de forma consecutiva, el pipeline de CD debe revertir automáticamente el tráfico a la última versión estable (imagen de Docker anterior).
- **El peligro del Rollback de Base de Datos:** Si la nueva versión aplicó una migración destructiva en los datos, un rollback de código romperá el sistema viejo, las migraciones de datos deben ser reversibles (down scripts) o seguir la estrategia de cambios aditivos (nunca borrar columnas viejas hasta que la versión nueva esté 100% consolidada).

- **Política de Respaldos (La Regla 3-2-1):**

  Ningún plan de despliegue está completo sin una estrategia de backups para mitigar desastres de pérdida de datos:

  - **3 copias de datos** (La base de datos de producción y dos respaldos).
  - **2 medios diferentes** (Ej. Un disco local en el servidor y un almacenamiento de objetos en la nube como AWS S3).
  - **1 copia fuera de la ubicación principal** (En una región geográfica distinta del proveedor de nube) por si el centro de datos principal sufre un fallo físico catastrófico.

